Olu taimeris

Programmatūras projekta dokumentācija

SATURS

1. Problēmas izpēte un analīze....3
1.1. Problēmas skaidrojums....3
1.2. Izpētes metodes....3
1.3. Mērķauditorija....3
1.4. Proprogrammatūras projekta apraksts....3
2. Programmatūras prasību specifikācija....4
2.1. Funkcionālās prasības....4
2.2. Nefunkcionālās prasības....4
3. Programmatūras projekta izstrādes plāns....4
3.1. Izvēlētā izstrādes modeļa pamatojums....4
4. Programmatūras projektējums....5
4.1. Tehnoloģiju apraksts....5
4.2. Struktūrskice....6
4.3. Funkciju apraksts....7
5. Lietotāja ceļvedis....8
1. pielikums - programmas kods....8

1. Problēmas izpēte un analīze

1.1. Problēmas skaidrojums

Es un mani draugi vāram olas samērā bieži, jo tās mums garšo, bet bieži vien prasās dažādu dzeltenuma konsistenci un ir slinkums katru reizi izdomāt cik ilgi vārīt.

1.2. Izpētes metodes

Lai veiktu situācijas izpēti tika lietotas divas izpētes metodes:

1. **Aptauja -** aprunājos ar draugiem par to vai viņiem arī slinkums likt taimeri pašiem.
2. **Interneta resursu atsauce** - atcerējos, ka dažus mēnešus atpakaļ, redzēju, kāda cilvēka video, kur pats uzprogrammēja olu taimeri.

1.3. Mērķauditorija

Lietotnei būs viena veida lietotājs:

1. Cilvēks, kurš bieži vāra olas.

1.4. Proprogrammatūras projekta apraksts

Projekta mērķis ir izstrādāt lietotni olas vārīšanai, kurā cilvēks, kas vāra olas, var nospiest pogu, kura attiecīga olas dzeltenuma konsistencei. Kā arī, ļauj izklaidēties, kamēr gaida taimera beigas.

2. Programmatūras prasību specifikācija

2.1. Funkcionālās prasības

1. Olu taimera ielāde
    - Redzamas trīs pogas - mīksts, vidējs, ciets
    - Nosaukums - “Olu taimeris”
2. Spēles laukuma ielāde
    - Spēles laukuma ielāde
3. Spēles punktu skaitītājs
    - Katru reizi, kad no spēlētāja 1 atsitās bumba, un spēlētājs 2 (dators) to neatsit atpakaļ, spēlētājs 1 saņem vienu punktu. Uzvar tas, kuram taimera beigās ir viss vairāk punkti.
4. Taimera beigu skaņa
    - Atskaņo skaņu, kad taimeris beidzās.

2.2. Nefunkcionālās prasības

1. Programmu paredzēts izstrādāt kā darbvirsmas programmatūru ar grafisko saskarni, kas tiek palaista mazā logā.
2. Programmu paredzēts darbināt uz datora, kas aprīkots ar operētājsistēmu Windows 10 vai jaunāku tās versiju.
3. Programmu paredzēt veidot ar vienkāršu, saskaņotu dizainu.
4. Taimera beigās, izveidot tā, lai var sākt jaunu taimeri pēc pirmā taimera beigām, kas arī atjauno spēles rezultātu uz 0:0.

3. Programmatūras projekta izstrādes plāns

3.1. Izvēlētā izstrādes modeļa pamatojums

Neskatoties uz apstākli, ka pirmajā mirklī varētu šķist, ka programmas prasības ir skaidras un saprotamas un projektu varētu realizēt izmantojot ūdenskrituma modeli, tomēr projekta realizācijai tiks izvēlēta agile metodika, jo ņemot vērā, ka izstrādes komanda nav ar iepriekšeju pieredz programmatūras projektu izstrādē, programmatūras izstrāde tiks sadalīta vairākos sprintos (viktorīnas pamata funkcionalitātes nodrošināšana, viktorīnas jautājumu un atbilžu sajaukšna, jautājumu un atbilžu ielašīšana no jautājumu bankas), lai pēc katra sprinta varētu atrādīt paveikto pasūtūtītājam, un katrā nākošajā sprintā varētu papildināt lietotnes funkcionalitāti.

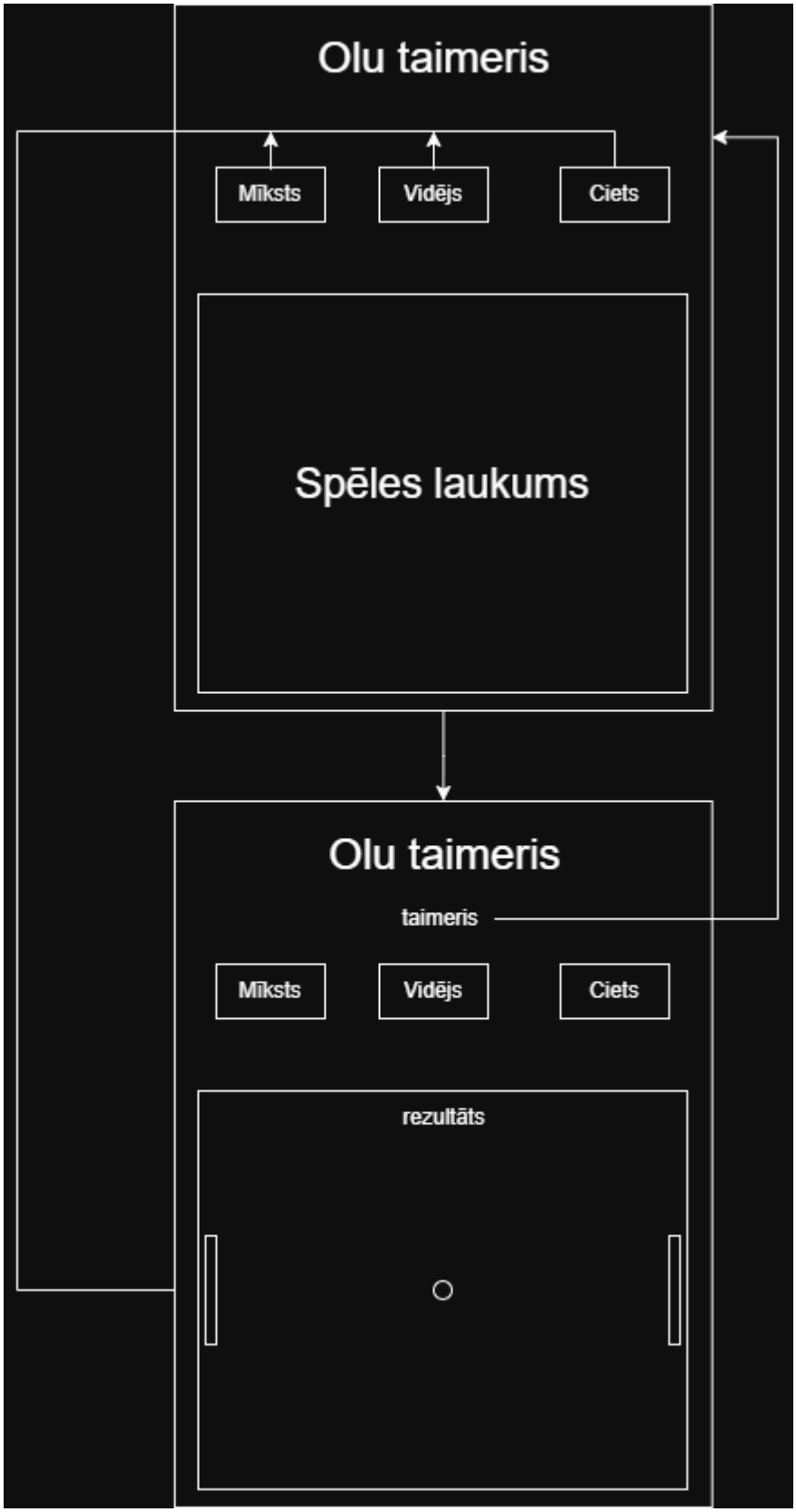
4. Programmatūras projektējums

4.1. Tehnoloģiju apraksts

Tā kā jāizstrādā darbvirsmas lietotne un izstrādes komandai ir neliela pieredze darbā ar programmēšanas valodu Python un grafiskās saskarnes veidošanu izmantojot Tkinter bibliotēku, tad

- programma tiks rakstīta programmēšanas valodā Python izmantojjot Tkinter bibliotēku;
- programma izstrādei tiks izmantota izstrādes vide Visual Studio Code;

4.2. Struktūrskice



4.3. Funkciju apraksts

- **play_beep()**

Pēc taimera beigām atskaņo skaņas signālu.

- **pong_draw_idle()**

Izveido Pong spēles laukumu.

- **pong_start()**

Sāk pong spēli, kad sākas, jebkurš taimeris.

- **pong_stop()**

Apstādina pong spēli, kad beidzas taimeris.

- **pong_move_paddles()**

Pārvieto spēlētāju platformiņas, jeb raketes.

- **pong_loop()**

Pong spēles kadrs, pārbauda bumbiņas un rakešu atrašanās vietu.

- **pong_draw_frame()**

Zīmē spēles kadru.

- **format_time()**

Pārvērš sekundes uz MM:SS formātu.

- **start_timer()**

Sāk taimeri.

- **stop_timer()**

Aptur taimeri.

- **tick()**

Katru sekundi, atjauno taimeri.

- **finish_timer()**

Apstāj taimeri, kad tas sasniedz nulli.

- **toggle_timer()**

Kad lietotājs nospiež pogu, pārbauda, ja taimeris darbojas - aptur, ja nedarbojas - sāk.

5. Lietotāja ceļvedis

Programmas mērķis: Olu taimeris, domāts tiem, kam slinkums katru reizi, kad vāra olas, manuāli likt taimeri. Ļauj arī izklaidēties kamēr gaida taimera beigas.

Programmas lietošanas instrukcija viktorīnas vadītājam

1. **Izdomā kādu konsistenci tu vēlies savām olām.**
2. **Nospied attiecīgo pogu.**
3. **Spēlē pong, tu esi zilā rakete, kusties uz augšu spiežot W un uz leju spiežot S. Uzvar tas, kuram ir viss vairāk punkti, kad beidzās taimeris.**

1. pielikums - programmas kods

```
import tkinter as tk
import math
import random
import winsound

# iestatījumi
SOFT_BOILED_SECONDS = 4 * 60
MEDIUM_BOILED_SECONDS   = 7 * 60
HARD_BOILED_SECONDS= 12 * 60

COLOR_BG = "#F9E4CD"
COLOR_CARD   = "#DEDEDD"
COLOR_SOFT = "#E8860D"
COLOR_MEDIUM     = "#EDA23A"
COLOR_HARD = "#EFE166"
COLOR_TEXT = "#000000"
COLOR_DIM  = "#B8A9A0"
COLOR_DONE = "#A7E8BD"
COLOR_PONG_BG= "#FFFFFF"
```

```
FONT_TITLE = ("Helvetica", 20, "bold")
FONT_MAIN = ("Helvetica", 12)
FONT_TIMER= ("Helvetica", 32, "bold")


def play_beep():
  winsound.MessageBeep(winsound.MB_ICONEXCLAMATION)

# pong spēle

PONG_WIDTH = 360
PONG_HEIGHT = 220
PADDLE_W = 8
PADDLE_H  = 40
BALL_RADIUS = 6
BALL_SPEED = 4.5

pong_running = False
pong_job = None

ball_x = PONG_WIDTH / 2
ball_y = PONG_HEIGHT / 2
ball_dx = BALL_SPEED
ball_dy = BALL_SPEED

left_paddle_y = PONG_HEIGHT // 2 - PADDLE_H // 2
right_paddle_y  = PONG_HEIGHT // 2 - PADDLE_H // 2

score = [0, 0]
```

```python
keys = {"w": False, "s": False}



def on_key_press(event):
    if event.keysym.lower() == "w":
        keys["w"] = True
    elif event.keysym.lower() == "s":
      keys["s"] = True



def on_key_release(event):
    if event.keysym.lower() == "w": keys["w"] = False
    elif event.keysym.lower() == "s":
        keys["s"] = False



def pong_start():
    global pong_running, ball_x, ball_y, ball_dx, ball_dy, score
    if pong_running:
        return
    pong_running = True
    score = [0, 0]
    ball_x = PONG_WIDTH / 2
    ball_y = PONG_HEIGHT / 2
    angle = random.uniform(-0.5, 0.5)
    direction = random.choice([-1, 1])
    ball_dx = BALL_SPEED * direction
    ball_dy  = BALL_SPEED * math.sin(angle)
    pong_loop()
```

```python
def pong_stop():
    global pong_running, pong_job
    pong_running = False
    if pong_job: canvas.after_cancel(pong_job)
    pong_job = None




def pong_move():
    global left_paddle_y, right_paddle_y
    speed = 6
    if keys["w"]:
        left_paddle_y = max(0, left_paddle_y - speed)
    if keys["s"]:
      left_paddle_y = min(PONG_HEIGHT - PADDLE_H, left_paddle_y + speed)
    # AI
    ai_speed = 4
    center = right_paddle_y + PADDLE_H / 2
    if center < ball_y:
        right_paddle_y += ai_speed
    elif center > ball_y:
        right_paddle_y -= ai_speed



def pong_loop():
    global ball_x, ball_y, ball_dx, ball_dy, score, pong_job

    if not pong_running:
        return
```

```
    pong_move()
    ball_x += ball_dx
    ball_y += ball_dy

    if ball_y <= 0 or ball_y >= PONG_HEIGHT: ball_dy *= -1

    if ball_x <= 10 and left_paddle_y <= ball_y <= left_paddle_y + PADDLE_H:
        ball_dx = abs(ball_dx)
     if ball_x >= PONG_WIDTH - 10 and right_paddle_y <= ball_y <= right_paddle_y +
PADDLE_H:
        ball_dx = -abs(ball_dx)

    if ball_x < 0:
      score[1] += 1
      reset_ball()
    if ball_x > PONG_WIDTH:
        score[0] += 1
        reset_ball()

    draw_pong()
    pong_job = canvas.after(16, pong_loop)


def reset_ball():
  global ball_x, ball_y, ball_dx
  ball_x, ball_y = PONG_WIDTH/2, PONG_HEIGHT/2
  ball_dx *= -1


def draw_pong():
```

```
    canvas.delete("all")
        canvas.create_rectangle(1,  1,  PONG_WIDTH,  PONG_HEIGHT,  fill=COLOR_PONG_BG,
outline="")
    canvas.create_rectangle(
        5, left_paddle_y, 5+PADDLE_W, left_paddle_y+PADDLE_H, fill="#00AEFF")
    canvas.create_rectangle(PONG_WIDTH-5-PADDLE_W, right_paddle_y,
                            PONG_WIDTH-5, right_paddle_y+PADDLE_H, fill="#FF3300")
    canvas.create_oval(ball_x-6, ball_y-6, ball_x+6, ball_y+6, fill="#FFFFFF")
    canvas.create_text(PONG_WIDTH/2-20, 10, text=score[0], fill=COLOR_DIM)
    canvas.create_text(PONG_WIDTH/2+20, 10, text=score[1], fill=COLOR_DIM)

## taimeris

timers = {
    "mīksts":   {"name": "Mīksti", "time": SOFT_BOILED_SECONDS,   "color": COLOR_SOFT},
    "vidējs": {"name": "Vidēji","time": MEDIUM_BOILED_SECONDS, "color": COLOR_MEDIUM},
    "ciets":   {"name": "Cieti",  "time": HARD_BOILED_SECONDS, "color": COLOR_HARD},
}


def start_timer(seconds):
    global remaining
    pong_stop()
    remaining = seconds
    update_timer()
    pong_start()

def update_timer():
    global remaining
    if remaining <= 0:
```

```
        label.config(text="GATAVS!")
        play_beep()
        pong_stop()
        return
    mins = remaining // 60
    secs = remaining % 60
    label.config(text=f"{mins:02}:{secs:02}")
    remaining -= 1
    window.after(1000, update_timer)

# UI ---------------------------------------------------

window = tk.Tk()
window.title("Olu taimeris")
window.configure(bg=COLOR_BG)
window.bind("<KeyPress>",   on_key_press)
window.bind("<KeyRelease>", on_key_release)
window.focus_set()

tk.Label(window, text="Olu taimeris",
         font=FONT_TITLE, bg=COLOR_BG, fg=COLOR_TEXT).pack(pady=10)

label = tk.Label(window, text="00:00", font=FONT_TIMER, bg=COLOR_BG, fg=COLOR_TEXT)
label.pack()

frame = tk.Frame(window, bg=COLOR_BG)
frame.pack(pady=10)

for key in timers:
  t = timers[key]
```

```
  tk.Button(frame,
            text=t["name"],
            bg=t["color"], fg=COLOR_TEXT,
            font=FONT_MAIN, width=10,
            command=lambda s=t["time"]: start_timer(s)
            ).pack(side="left", padx=5)

canvas = tk.Canvas(window, width=PONG_WIDTH, height=PONG_HEIGHT,
                   bg=COLOR_PONG_BG, highlightthickness=0)
canvas.pack(pady=10)

tk.Label(window, text="Zilais taisnstūris, W un S lai kustētos",
         font=("Helvetica", 9), bg=COLOR_BG, fg=COLOR_TEXT).pack(pady=5)

window.mainloop()
```